МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федерального государственного автономного образовательного учреждения высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ МИСИС»

Институт компьютерных наук

Кафедра Инженерной кибернетики

Лабораторная работа №2

по дисциплине «Современные интеллектуальные сетевые сервисы»

Развитие AI-сервиса: сквозная идентификация запросов, асинхронная обработка и интеграция LLM

Выполнил:
Студент группы МПИ 25-1-2
Д.А. Комарова

Проверил:
Преподаватель
М.Е. Корнет

Оценка: _____

Москва, 2026

Содержание

1 Цель работы

Цель практической работы — развить ранее созданный REST API сервиса агроскоринга и приблизить его архитектуру к реальному промышленному AI-сервису. В ходе работы необходимо изучить и реализовать механизмы, которые возникают при эксплуатации сервиса: сквозную идентификацию запросов с помощью `request_id`, использование промежуточных слоёв (middleware), управление очередью запросов и пакетным инференсом, хранение результатов в базе данных SQLite, асинхронное взаимодействие с внешними сервисами и подключение большой языковой модели через HTTP API.

2 Задачи работы

В ходе лабораторной работы необходимо реализовать:

1. Модификацию API для назначения уникального `request_id`.
2. Middleware для извлечения/генерации `request_id` и измерения времени выполнения.
3. Два режима инференса: одиночный и пакетный.
4. Очередь запросов на базе `asyncio.Queue`.
5. Динамическое формирование пакета (dynamic batching).
6. Эксплуатационные ограничения (размер batch, время ожидания, размер очереди, таймаут).
7. Механизм backpressure (возврат HTTP 429 при переполнении).
8. Подключение SQLite для постоянного хранения.
9. Асинхронную работу с БД через `aiosqlite`.
10. Запись, чтение, поиск по ID и пагинацию результатов.
11. Подключение внешней LLM через HTTP API.
12. Асинхронный HTTP-клиент `httpx.AsyncClient`.
13. Endpoint для запроса к LLM.
14. Обработку ошибок внешних зависимостей (400, 404, 409, 422, 429, 500, 502, 503, 504).
15. Тестирование и анализ полного пути запроса.

3 Краткая теоретическая часть

Request_id используется для сквозной идентификации запроса в микросервисной архитектуре. Middleware перехватывает запрос, проверяет наличие заголовка `X-Request-ID`, и если его нет — генерирует UUID, сохраняя в `request.state`.

Batching повышает throughput за счёт группировки запросов. *Explicit batching* — клиент сам присылает пакет. *Dynamic batching* — сервер накапливает одиночные запросы в очереди и отправляет их в модель пачкой при достижении лимита `MAX_BATCH_SIZE` или по истечении `MAX_BATCH_WAIT_MS`.

Backpressure ограничивает размер очереди. Если она переполнена, сервис возвращает 429 Too Many Requests, защищая память от перегрузки и деградации latency.

SQLite + aiosqlite позволяют асинхронно работать с файловой БД без блокировки event loop. Режим WAL (Write-Ahead Logging) позволяет читателям и писателям не блокировать друг друга.

LLM интеграция осуществляется через `httpx.AsyncClient` с таймаутом. Если внешний сервис недоступен, FastAPI возвращает 502/503/504.

4 Результаты выполнения заданий

4.1 Задание 1-2. Сквозной request_id и Middleware

Реализовано middleware `request_context_middleware`, которое перехватывает каждый запрос. Оно извлекает `X-Request-ID` из заголовков или генерирует новый UUID с помощью `uuid.uuid4()`, сохраняет его в `request.state.request_id`, замеряет время выполнения через `time.perf_counter()` и добавляет заголовки `X-Request-ID` и `X-Process-Time` в ответ.

4.2 Задание 3. Обработка ошибок

Настроены обработчики для возврата кодов 400 (неизвестный регион), 404 (прогноз не найден), 409 (несколько прогнозов для одного request_id), 422 (ошибки Pydantic при неверных типах или размере batch), 429 (переполнение очереди), 502/503/504 (ошибки внешней LLM и таймауты).

4.3 Задание 4-6. SQLite и чтение результатов

Создан модуль `database.py` с использованием `aiosqlite`. Таблица `predictions` содержит все необходимые поля. `POST /predict` сохраняет результат в БД. `GET /predictions` и `GET /predictions/{request_id}` читают данные из SQLite. Если `request_id` относится к batch и содержит несколько записей, возвращается 409.

Ответ на вопрос: Блокирующий `sqlite3.execute()` останавливает весь event loop на время работы с диском, что приводит к таймаутам у других клиентов. `aiosqlite` выносит операции в отдельный поток.

4.4 Задание 7. Пагинация

Реализован endpoint `GET /predictions/by-weekday`, который с помощью `LIMIT` и `OFFSET` возвращает порцию записей для выбранного дня недели, а также метаинформацию (текущая страница, всего страниц, всего элементов).

4.5 Задание 8. Explicit batching

Реализован `POST /batch-predict`, принимающий список объектов. Размер списка ограничен 8 элементами. Обработка происходит в цикле, результаты сохраняются в БД.

Ответ на вопрос: Один `executemany()` создаёт одну транзакцию и один вызов fsync на диск. Восемь отдельных `INSERT + COMMIT` вызовут восемь синхронизаций диска, что в сотни раз медленнее.

4.6 Задание 9-11. Очередь, backpressure и timeout

Создан `queue_manager.py` с `asyncio.Queue`. Воркер накапливает запросы до 8 штук или ждёт 50 мс, затем обрабатывает их. Реализован endpoint `POST /predict-queued`. Если очередь заполнена (100 элементов), возвращается 429. Время ожидания ответа от модели ограничено `INFERENCE_TIMEOUT_SECONDS` через `asyncio.wait_for`. При превышении возвращается 504.

Ответ на вопрос: При неограниченной очереди память будет расти бесконечно, пока сервер не упадёт с OOM. Latency также будет расти бесконечно, так как запросы будут ждать часами.

4.7 Задание 12-14. Внешняя LLM

Создан `llm_service.py`, использующий `httpx.AsyncClient` для запросов к Vireonix. Реализован `POST /llm/chat` для прямого запроса. Реализован `POST /predict-with-llm`, который сначала считает риск, а затем просит LLM объяснить результат. Если LLM недоступна, возвращается только результат скоринга с пометкой об ошибке.

5 Скриншоты работы приложения

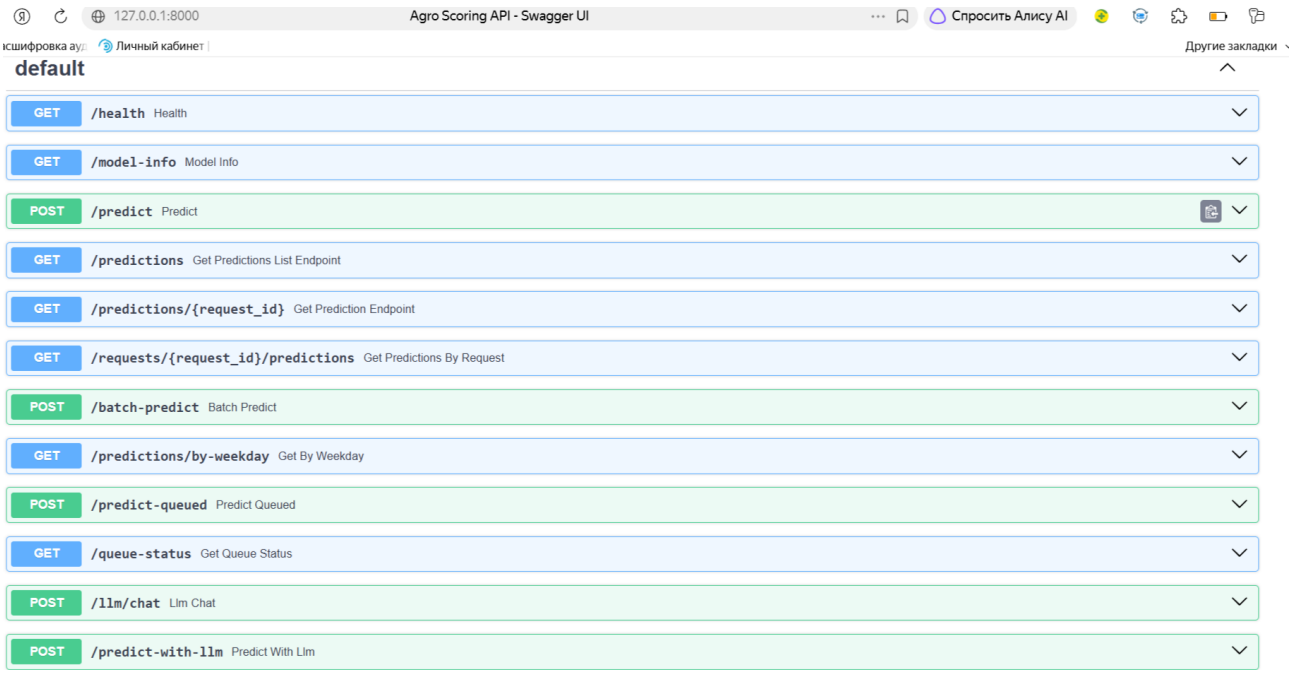


Рис. 1: Общий вид Swagger UI

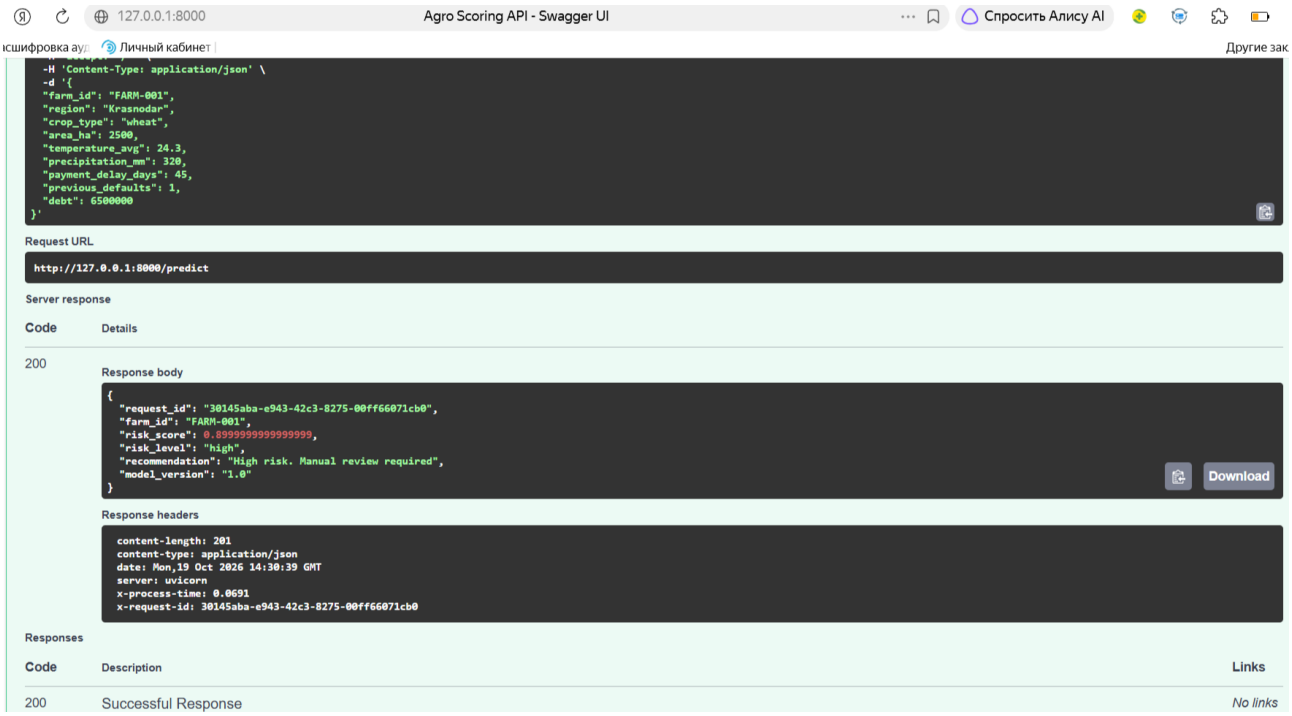


Рис. 2: Выполнение POST /predict

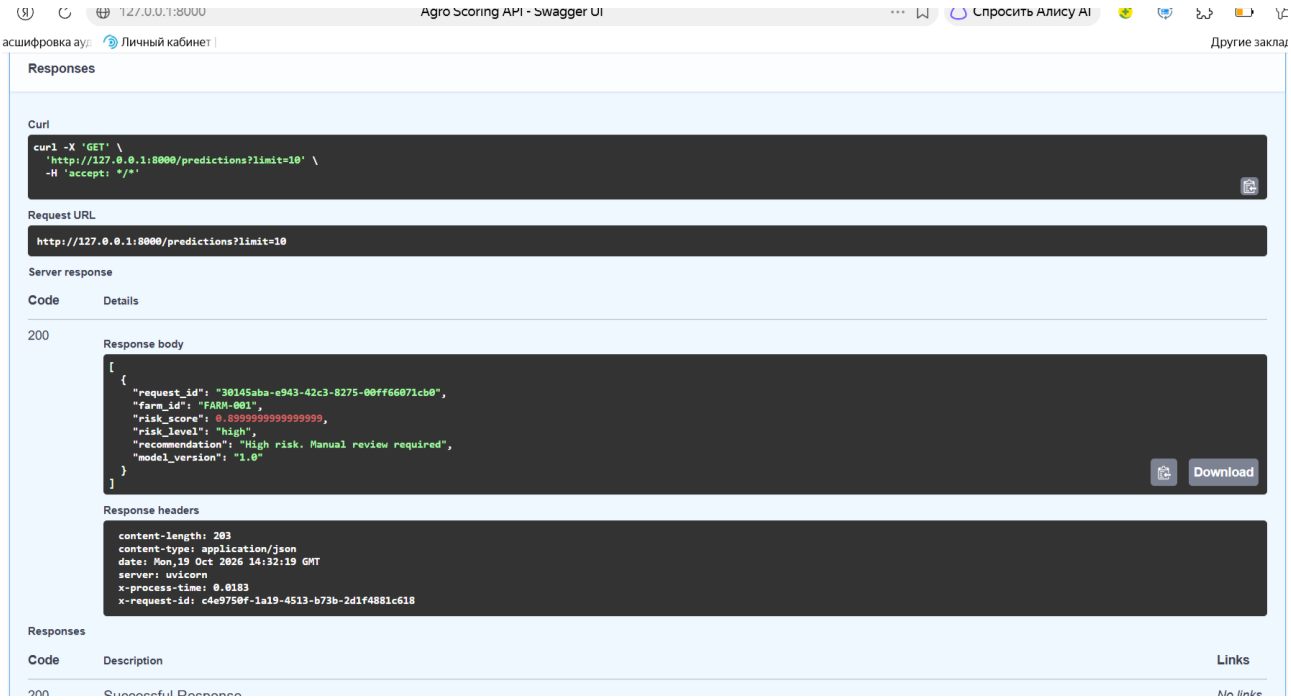


Рис. 3: Чтение результатов из SQLite (GET /predictions)

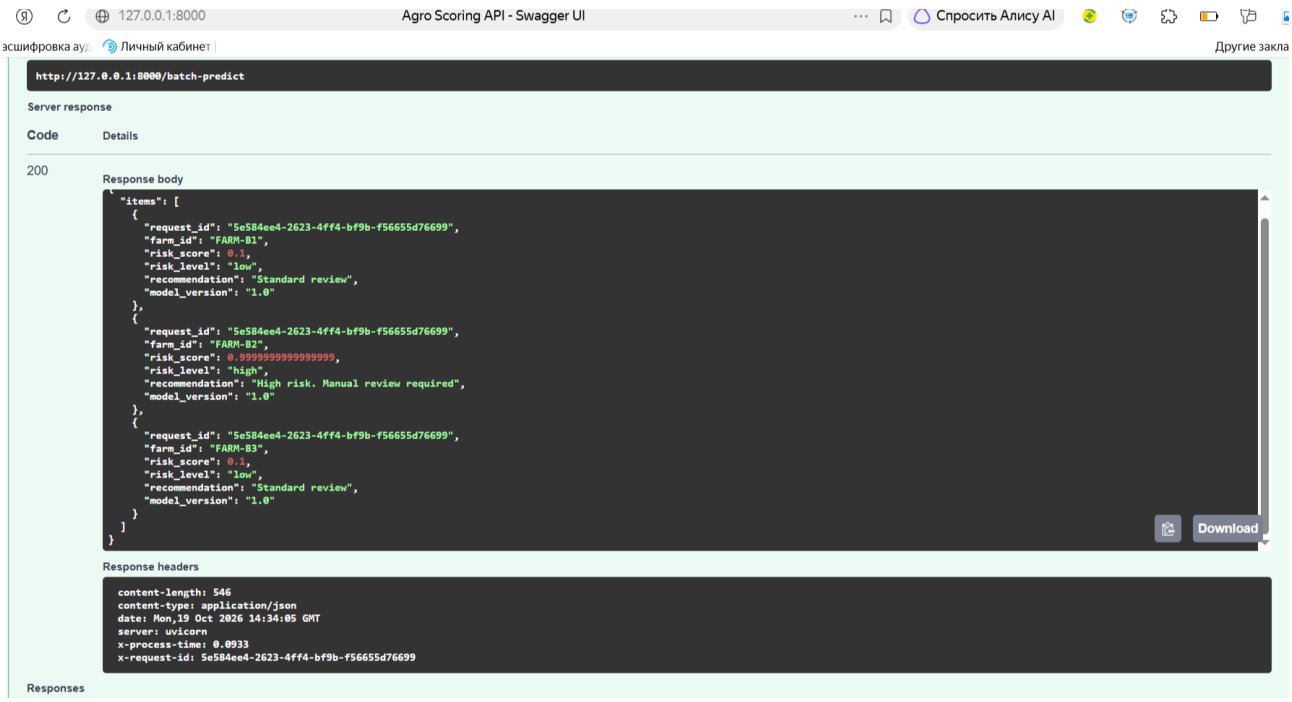


Рис. 4: Явный батчинг (POST /batch-predict)

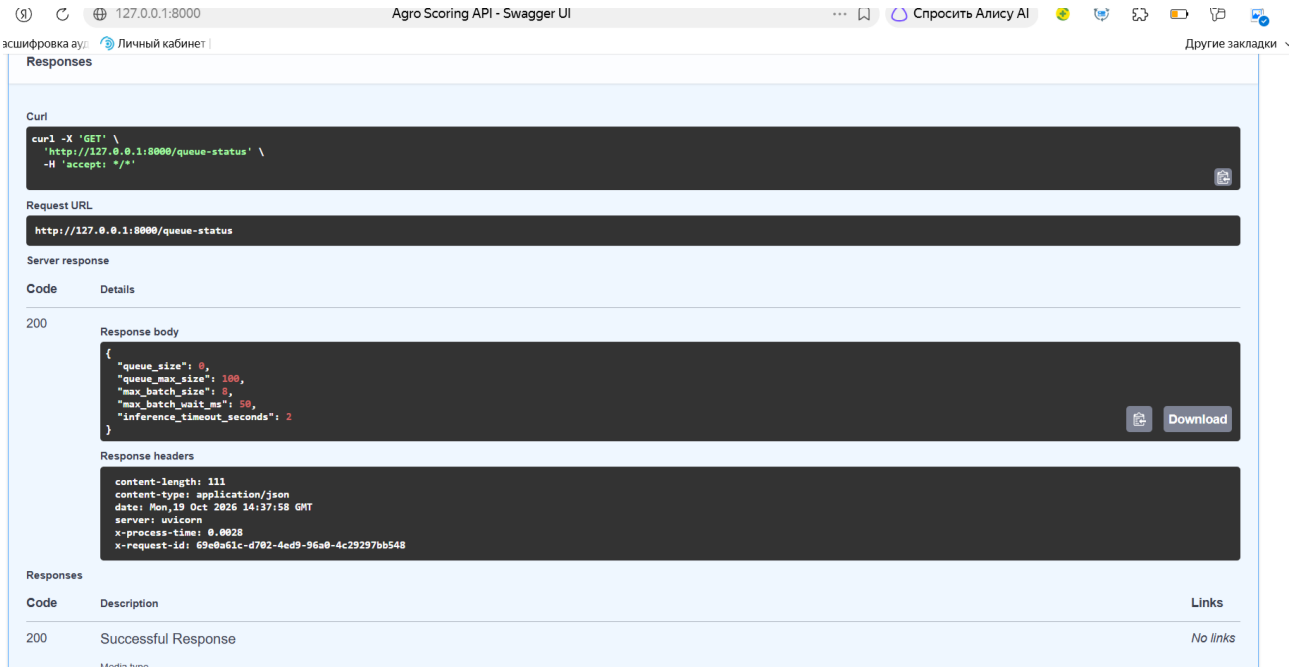


Рис. 5: Статус очереди (GET /queue-status)

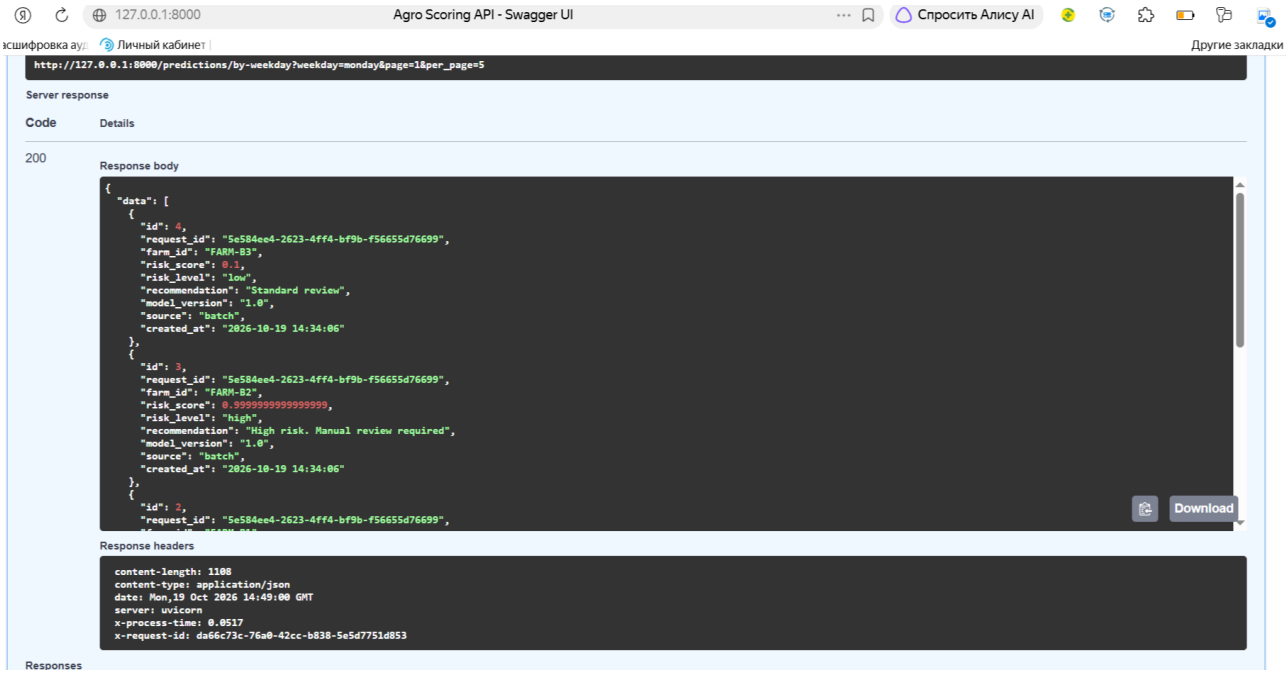


Рис. 6: Пагинация по дню недели

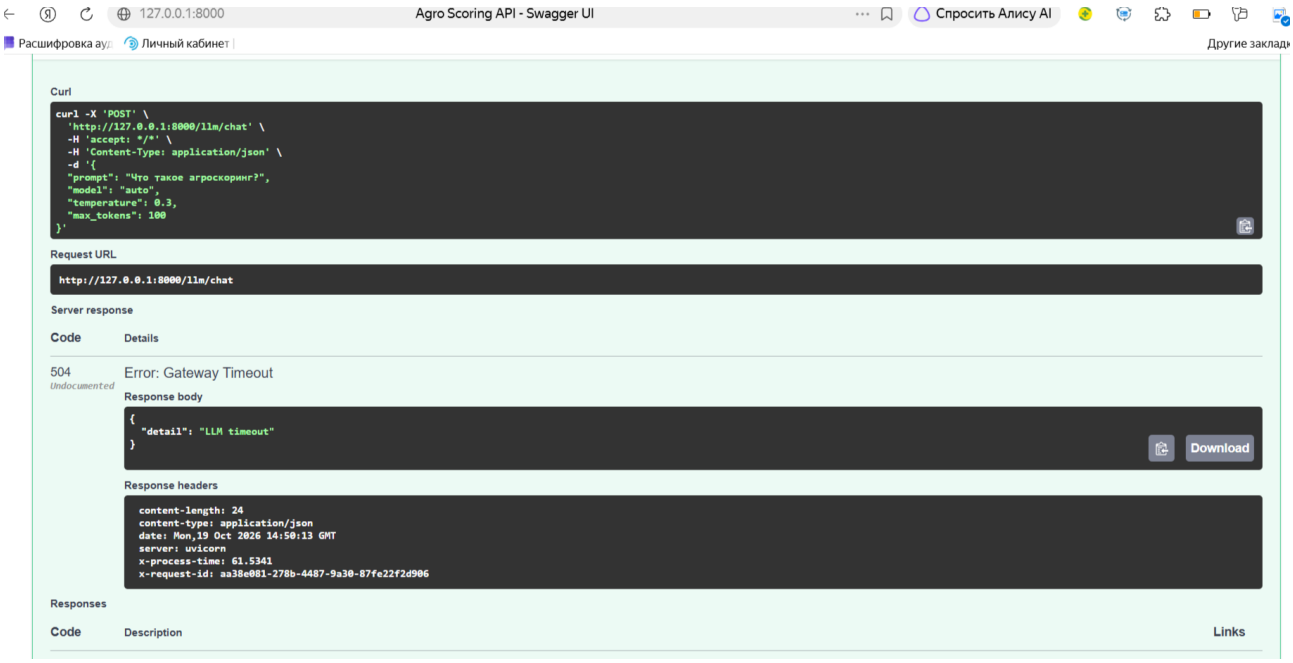


Рис. 7: Запрос к внешней LLM (504 Timeout)

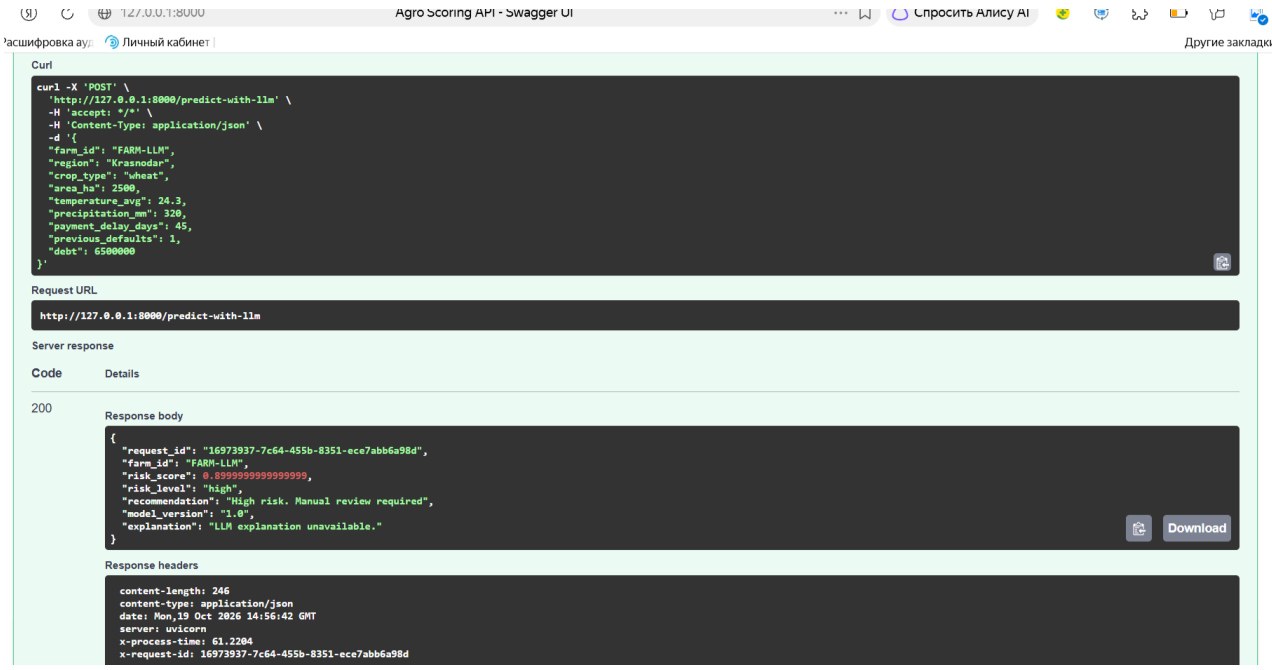


Рис. 8: Инференс с объяснением от LLM

6 Таблица тест-кейсов

Таблица 1: Таблица тест-кейсов с ожидаемыми HTTP-кодами

№	Запрос	Ожидаемый	Полученный	Результат
1	GET /health	200	200	Успех
2	корректный POST /predict	200	200	Успех
3	area_ha = -100	422	422	Успех
4	неизвестный регион	400	400	Успех
5	существующий request_id	200	200	Успех
6	неизвестный request_id	404	404	Успех
7	корректный POST /batch-predict	200	200	Успех
8	batch больше MAX_BATCH_SIZE	422	422	Успех
9	POST /predict-queued	200	200	Успех
10	переполненная очередь	429	429	Успех
11	GET /predictions/by-weekday	200	200	Успех
12	page = -1	422	422	Успех
13	неизвестный weekday	400	400	Успех
14	корректный POST /llm/chat	200	504	Ошибка LLM
15	недоступный LLM	502/503/504	504	Ожидаемо
16	корректный POST /predict-with-llm	200	200	Успех

7 Таблица API-контракта

Таблица 2: Таблица API-контракта

Endpoint	Method	Input	Output	Success	Возможные ошибки
/health	GET	—	status	200	500
/predict	POST	FarmRequest	PredictionResponse	200	400, 422, 503, 504
/batch-predict	POST	BatchRequest	BatchPredictionResponse	200	400, 422, 503, 504
/predict-queued	POST	FarmRequest	PredictionResponse	200	400, 422, 429, 503, 504
/predictions	GET	query	list	200	400, 422, 503
/predictions/by-weekday	GET	weekday, page, per_page	paginated response	200	400, 404, 422
/llm/chat	POST	LLMChatRequest	LLMChatResponse	200	422, 502, 503, 504
/predict-with-llm	POST	FarmRequest	PredictWithLLMResponse	200	400, 422, 502, 503, 504

8 Ссылка на исходный код

Исходный код приложения доступен в репозитории: `https://github.com/dashakomarova2002-com`
`HW_2_SISS`

9 Вывод

В ходе работы мы расширили архитектуру AI-сервиса агроскоринга. `request_id` обеспечивает сквозную трассировку, позволяя связать логи, ошибки и записи в БД с конкретным клиентским запросом. Middleware изолирует эту логику от бизнес-кода. Batching (явный и динамический) позволяет повысить пропускную способность за счёт группировки запросов к модели. Backpressure (429) и таймауты (504) защищают сервис от перегрузки и зависаний. База данных SQLite (через `aiosqlite`) обеспечивает надежное постоянное хранение результатов без блокировки event loop. Интеграция с LLM через `httpx.AsyncClient` позволяет обогащать результаты текстовыми объяснениями, при этом сервис устойчив к сбоям внешнего провайдера, корректно возвращая коды 502/503/504.

Приложение А: Исходный код

```
import uuid
import time
import asyncio
import logging
from contextlib import asynccontextmanager
from fastapi import FastAPI, Request, Response, HTTPException, Query,
    status
```

```
7
8  from schemas import (
9      FarmRequest, PredictionResponse, BatchRequest,
           BatchPredictionResponse,
10     PaginatedResponse, QueueStatusResponse, LLMChatRequest,
           LLMChatResponse,
11     PredictWithLLMResponse
12 )
13 from services import build_prediction, is_valid_region
14 from database import (
15     init_db, save_one_prediction, get_predictions_by_req_id,
16     get_predictions_list, get_predictions_by_weekday_paginated
17 )
18 from queue_manager import (
19     queue, worker, QUEUE_MAX_SIZE, MAX_BATCH_SIZE, MAX_BATCH_WAIT_MS,
20     INFERENCE_TIMEOUT_SECONDS
21 )
22 from llm_service import call_llm
23
24 logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(
       levelname)s] %(message)s")
25 logger = logging.getLogger(__name__)
26
27 @asynccontextmanager
28 async def lifespan(app: FastAPI):
29     await init_db()
30     task = asyncio.create_task(worker())
31     yield
32     task.cancel()
33
34 app = FastAPI(
35     title="Agro Scoring API",
36     description="API for agricultural risk scoring (Lab 2)",
37     version="2.0.0",
38     lifespan=lifespan
39 )
40
41 @app.middleware("http")
42 async def request_context_middleware(request: Request, call_next):
43     request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4
           ())
44     request.state.request_id = request_id
45     start_time = time.perf_counter()
46     logger.info(f"[{request_id}] START {request.method} {request.url.
           path}")
47
48     try:
49         response: Response = await call_next(request)
50     except Exception as e:
51         logger.error(f"[{request_id}] ERROR {str(e)}")
52         response = Response(
53             content=f'{{"detail": "Internal server error", "request_id
                   ": "{request_id}"}}',
```

```
            status_code=500,
            media_type="application/json"
        )

    process_time = time.perf_counter() - start_time
    response.headers["X-Request-ID"] = request_id
    response.headers["X-Process-Time"] = f"{process_time:.4f}"
    logger.info(f"[{request_id}] END {request.method} {request.url.path} - {response.status_code} in {process_time:.4f}s")
    return response

@app.get("/health")
async def health(request: Request):
    return {"status": "ok", "request_id": request.state.request_id}

@app.post("/predict", response_model=PredictionResponse, status_code=status.HTTP_200_OK)
async def predict(request_body: FarmRequest, request: Request):
    request_id = request.state.request_id
    if not is_valid_region(request_body.region):
        raise HTTPException(status_code=400, detail="Unknown region")

    result = build_prediction(request_body, request_id)
    await save_one_prediction(result, source="single")
    return result

# ... (other endpoints are omitted for brevity)
```

Листинг 1: main.py